EduGenie: Google Gemini Powered Learning Assistant

Project Report

Detail	Information
Project title	EduGenie: Google Gemini Powered Learning Assistant
Submitted by	[Enter your name]
Register number	[Enter your register number]
Institution / department	[Enter institution and department]
Mentor / guide	[Enter mentor name]
Submission date	[Enter submission date]
Source repository	https://github.com/Maxiii-17/EduGenic
Document version	1.0 — September 27, 2026

This report describes the current Gemini-based implementation and its local execution workflow. AI model access may change independently of the source code.

Contents

1. Abstract
2. Introduction and problem statement
3. Objectives and scope
4. Requirements and technology stack
5. System architecture
6. Module descriptions
7. API specification
8. User interface and workflow
9. Installation and execution
10. Testing and evaluation
11. Challenges and solutions
12. Limitations
13. Future enhancements
14. Conclusion
15. References
16. Demonstration and viva guide

1. Abstract

EduGenie is a web-based educational assistant that uses generative AI to support independent learning. The application provides five learning tools: question answering, concept explanation, passage summarization, multiple-choice quiz generation, and learning-path recommendations. Students enter a question, topic, or passage in a browser and receive a response beneath the relevant form without reloading the page.

The backend is implemented in Python using FastAPI. A lightweight HTML, CSS, and JavaScript interface communicates with the backend through HTTP requests. Google Gemini generates educational content through the Google Gen AI SDK. An optional local LaMini-Flan-T5 model is implemented for concept explanation, with Gemini used when the local model is unavailable.

The project demonstrates modular backend design, prompt construction, JSON processing, and browser-to-server integration. It offers a practical learning prototype while retaining limitations associated with model accuracy, API quotas, response latency, and basic input validation.

Keywords: generative AI, education, FastAPI, Google Gemini, quiz generation, summarization, learning assistant.

2. Introduction and Problem Statement

Students often consult several tools to answer questions, understand difficult concepts, prepare revision notes, practice questions, and plan further study. Searching across these resources can interrupt learning and make it difficult to identify a useful starting point.

EduGenie brings these activities into one simple interface. Its purpose is to reduce the effort required to obtain an initial explanation, revision summary, practice quiz, or study plan. Learners remain responsible for checking important facts against reliable educational sources.

Example scenarios

- **General knowledge:** A student asks, "Which is the largest ocean?" and receives an answer identifying the Pacific Ocean.
- **Self-assessment:** A student enters "Pythagorean theorem," generates a quiz, and checks each selected answer.
- **Study planning:** A learner enters "SQL" and receives a suggested sequence of beginner, intermediate, and advanced topics with timelines and resources.

3. Objectives and Scope

Objectives

1. Provide a single web interface for five common study tasks.
2. Generate understandable educational explanations and summaries.
3. Produce practice MCQs with immediate answer feedback.
4. Suggest ordered learning topics and useful resources.
5. Keep the core installation lightweight by using cloud inference.
6. Organize each feature in a separate Python module for maintainability.
7. Support straightforward local execution through VS Code or a Windows launcher.

Implemented scope

- Text-based questions, topics, and passages.
- Five feature forms, loading messages, result containers, and quiz feedback.
- FastAPI endpoints, a basic health endpoint, and generated API documentation.
- Server-side Gemini credentials and configurable primary model.
- Sequential fallback between configured Gemini models for selected failures.
- Optional local-model explanation logic.

Scope boundaries

The current application has no accounts, database, saved history, PDF upload, voice interface, persistent progress tracking, or teacher dashboard. Learning recommendations are based on the submitted topic; there is no stored learner profile or adaptive scoring system. Ollama and Pollinations are not part of the current application.

4. Requirements and Technology Stack

Component	Purpose
Python 3.10+	Backend language; dependency support should be checked for the selected Python version
FastAPI	HTTP routes and automatic OpenAPI documentation
Uvicorn	Local ASGI application server
Jinja2	Serves the HTML template
HTML and CSS	Page structure, layout, forms, and responsive styling
JavaScript Fetch API	Sends requests and updates results without a page refresh
Google Gen AI SDK (`google-genai`)	Communicates with Gemini
`python-dotenv`	Loads local environment configuration
Git and GitHub	Source control and repository hosting
Transformers and PyTorch (optional)	Local LaMini concept explanations

Operating requirements

For cloud inference, the computer needs Python, a browser, internet connectivity, and a valid Gemini API key with available quota. No local GPU is required for this mode. The application runs locally and is accessed through a browser.

Local LaMini inference requires additional model downloads, compatible machine-learning dependencies, and sufficient RAM and disk space. Those requirements are significantly larger than the cloud-only installation. Hardware performance measurements have not been established for this project.

5. System Architecture

Request flow

```
Student using a web browser
          |
          | HTML/CSS interface and JavaScript fetch
          v
FastAPI application (main.py)
          |
          +-- /qa ------------------> qna.py
          +-- /explain/ ------------> explanation_module.py
          +-- /summarize/ ----------> summary_module.py
          +-- /quiz ----------------> quiz_module.py
          +-- /learn/recommendations -> learning_path.py
                                        |
                         Google Gemini client
                         (gemini_client.py)
                                        |
                         Google Gemini API
                                        |
                        Text / JSON response
                                        |
                        Browser result display

Explanation only: optionally try local LaMini before Gemini.
```

Processing sequence

1. FastAPI serves `templates/index.html` and the stylesheet.
2. The learner submits one of the feature forms.
3. JavaScript displays a loading message and sends a GET or POST request.
4. FastAPI reads the input and calls the matching Python function.
5. The function constructs a prompt or passes the question to Gemini.
6. The Gemini client extracts the response text or tries a configured fallback model for selected failures.
7. Quiz output is cleaned and parsed as JSON; other outputs remain text.
8. The API returns JSON to the browser.
9. JavaScript updates the result container. Quiz answers are checked in the browser.

Core application structure

```
EduGenie Google Gemini Powered Learning Assistant/
|-- main.py
|-- gemini_client.py
|-- qna.py
|-- explanation_module.py
|-- quiz_module.py
|-- summary_module.py
|-- learning_path.py
|-- templates/
|   `-- index.html
|-- static/
|   `-- style.css
|-- requirements.txt
|-- run.bat
|-- .env.example
|-- .gitignore
|-- README.md
`-- PROJECT_REPORT.md
```

The local `.env` file holds configuration and is excluded from Git. It is not included in this report. There is no application database or database schema.

6. Module Descriptions

6.1 Question answering — `qna.py`

`answer_question_with_gemini(question)` passes the question to `generate_gemini_text()` and returns its response. Errors are caught and represented as a readable error string. The feature is useful for quick doubts and general educational questions, but it does not independently verify answers or provide guaranteed citations.

6.2 Concept explanation — `explanation_module.py`

`explain_topic(topic)` first attempts to load `MBZUAI/LaMini-Flan-T5-783M` if local explanations are enabled. Tokenization and generation occur locally when the model is available. The model is loaded lazily and retained in process memory after a successful load. Local generation is limited to 150 new tokens.

If dependencies, model loading, or local generation fail, the function requests a concise, school-student-friendly explanation from Gemini. Setting `EDUGENIE_EXPLAIN_LOCAL=0` disables the local attempt.

The demonstrated cloud workflow used Gemini fallback. Successful LaMini inference has not been verified in the current setup.

6.3 Summarization — `summary_module.py`

`summarize_text(text)` asks Gemini to summarize the supplied text in simple language. It is intended for revision of educational passages. The application does not independently measure factual coverage, compression ratio, or whether the summary is always shorter than the source.

6.4 Quiz generation — `quiz_module.py`

`generate_quiz(text)` requests a JSON array containing three MCQs. Each requested item has:

- `question`: the question text.
- `options`: four answer choices.
- `answer`: the exact text of the correct choice.

`clean_json_block()` removes matching Markdown JSON code fences before `json.loads()` parses the result. The current code checks that the parsed output is a list, but does not fully validate the number of questions, option counts, or answer membership.

The browser renders radio buttons and a **Check Answer** button for each question. A correct selection displays positive feedback; an incorrect selection displays the expected answer. This is a self-practice tool: answers are included in the browser response, so it is not a secure examination system.

6.5 Learning recommendations — `learning_path.py`

`get_learning_recommendations(topic)` constructs a prompt requesting levels, key topics, order of learning, estimated time, recommended resources, and four to six learning tips. It returns generated text rather than a structured progress-tracking object. Suggested resources and timelines require review by the learner.

6.6 Shared Gemini client — `gemini_client.py`

The client reads `GEMINI_API_KEY`, with `GOOGLE_API_KEY` as an alternative. The configured primary model defaults in code to `gemini-3.6-flash`. Current fallback identifiers are `gemini-3.5-flash`, `gemini-flash-latest`, and `gemini-3-flash-preview`.

The client attempts models sequentially for recognized model-not-found, quota, and overload failures. It returns the first non-empty response. These identifiers describe the current code configuration, not a guarantee of ongoing provider availability. Model access should be checked in the account before running the application.

Fallback does not provide unlimited quota. Limits depend on the provider, model, project, and billing tier; generating another API key in the same project does not create an independent quota allocation. The error helper currently mentions 20 requests/day, reflecting an observed limit during development rather than a universal Gemini limit.

7. API Specification

Method	Path	Input	Response field(s)
GET	`/`	None	HTML page
GET	`/health`	None	`status`
GET	`/qa`	Query parameter `question`	`answer`
POST	`/explain/`	JSON body with `topic`	`topic`, `explanation`
POST	`/summarize/`	JSON body with `text`	`summary`
POST	`/quiz`	JSON body with `text`	`quiz`
GET	`/learn/recommendations`	Query parameter `topic`	`topic`, `recommendation`
GET	`/static/style.css`	None	CSS stylesheet
GET	`/docs`	None	Interactive Swagger UI

Request examples

```
GET /qa?question=Which%20is%20the%20largest%20ocean%3F
```

Illustrative response; generated wording varies:

```
{"answer": "The Pacific Ocean is the largest ocean on Earth."}

POST /explain/
Content-Type: application/json

{"topic": "Photosynthesis"}

POST /summarize/
Content-Type: application/json

{"text": "Plants use sunlight, water, and carbon dioxide to produce sugars through
photosynthesis. Oxygen is released during this process."}

POST /quiz
Content-Type: application/json

{"text": "The Pythagorean theorem describes the sides of a right-angled triangle: a squared
```

```
plus b squared equals c squared."}

GET /learn/recommendations?topic=SQL
```

Validation and errors

- Missing or empty `topic`/`text` values in valid JSON objects return HTTP 400 on the three POST feature endpoints.
- Missing required GET query parameters are handled by FastAPI, normally with HTTP 422.
- Validation does not comprehensively cover whitespace-only values, input size, wrong JSON shapes, or field types.
- AI failures are usually returned inside a normal HTTP 200 response as an error string or a quiz error object. HTTP 200 alone is therefore insufficient to establish AI success.
- `/health` confirms that the application responds; it does not check Gemini credentials, quota, or generation.

8. User Interface and Workflow

The page has five separate forms, each with an input field, submit button, and result container. Styling uses a light background, rounded sections, blue buttons, and responsive layout rules. The current form inputs are single-line fields, including the passage fields.

Using the application

1. Open the application URL in a browser.
2. Choose the section for the required learning task.
3. Enter a question, topic, or passage.
4. Click the section's submit button.
5. Wait for the loading message to be replaced by a result or error.
6. For a quiz, select a radio option and click **Check Answer**.

Results are shown without a full page reload. Text responses preserve line breaks; Markdown headings and emphasis may appear as literal symbols because the frontend does not include a Markdown renderer. Results are not saved after a page refresh.

9. Installation and Execution

9.1 Open the folder in VS Code

Choose **File** → **Open Folder** and open:

```
E:\naan mudhalavan\EduGenie Google Gemini Powered Learning Assistant
```

Choose **Terminal** → **New Terminal**. If necessary, navigate to the folder:

```
cd "E:\naan mudhalavan\EduGenie Google Gemini Powered Learning Assistant"
```

9.2 Install dependencies

For an isolated installation, create a virtual environment and use its Python executable directly:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

If using the existing Python installation instead:

```
python -m pip install -r requirements.txt
```

9.3 Configure Gemini once

On a fresh checkout, copy `.env.example` to `.env` only if `.env` does not already exist. Edit it to contain your own values:

```
GEMINI_API_KEY=replace_with_your_own_key
GEMINI_MODEL=gemini-3.6-flash
EDUGENIE_EXPLAIN_LOCAL=0
```

Use a model available to your account. The restored `.env.example` still specifies an older model (`gemini-2.0-flash`), so review that value rather than assuming the template is current. The application code uses the primary model from the environment when one is provided.

The key is stored once on the server; students do not enter it for every question. Restart the server after changing environment configuration.

9.4 Start the server

For the virtual environment:

```
.\.venv\Scripts\python.exe -m uvicorn main:app --reload
```

For the existing Python installation:

```
python -m uvicorn main:app --reload
```

Open **http://127.0.0.1:8000**. A terminal showing “Uvicorn running” is normal: leave it open while using the application. Press **Ctrl+C** to stop it.

9.5 Windows shortcut

With dependencies installed in the Python environment on PATH, double-click `run.bat`. It starts Uvicorn and opens the browser after a short delay. The launcher does not install dependencies or select `.venv` automatically. If the page opens before startup completes, refresh it after the server is ready.

Opening `templates/index.html` directly is not the full application: the root-relative CSS and API URLs require the server. If port 8000 is occupied, stop the known earlier server or use `--port 8001` and open the matching address.

10. Testing and Evaluation

Evidence and status

During earlier development, live requests returned an ocean answer, a Photosynthesis explanation, an Industrial Revolution summary, three Pythagorean-theorem MCQs, and an SQL learning roadmap. Local HTTP checks exercised the homepage, stylesheet, and health endpoint.

These are historical smoke checks, not a comprehensive automated regression suite or a fresh availability check performed while writing this report. No measured accuracy percentage, uptime guarantee, concurrency benchmark, or response-time SLA has been established.

Repeatable acceptance checklist

ID	Test	Expected result
T01	Open `/`	All five feature sections appear
T02	Request `/static/style.css`	HTTP 200 with CSS content
T03	Request `/health`	`{"status":"ok"}`
T04	Ask “Which is the largest ocean?”	Response identifies the Pacific Ocean
T05	Explain Photosynthesis	Understandable explanation; review scientific accuracy
T06	Summarize a known passage	Relevant summary preserving key facts
T07	Generate a Pythagorean-theorem quiz	Three questions, four choices each, valid answers; verify manually
T08	Check a correct quiz selection	Positive feedback
T09	Check a wrong quiz selection	Correction displays the expected answer
T10	Request a SQL learning path	Ordered topics with timelines and resources
T11	Send empty `topic`/`text` to POST endpoints	HTTP 400
T12	Omit a required GET parameter	Validation response, normally HTTP 422
T13	Run with an unavailable API key	Understandable configuration/provider error
T14	Encounter quota or model overload	Configured fallback is attempted where recognized
T15	View on a narrow screen	Forms and results remain usable

Use `/docs` to submit API requests interactively. Inspect response content as well as status codes. Record actual results and screenshots during the final demonstration; cloud checks consume provider quota.

Screenshots to include in an academic submission

1. Homepage with all feature sections.
2. Question and answer for the largest-ocean scenario.
3. Concept explanation result.
4. Original passage and generated summary.
5. Generated quiz with correct and incorrect answer feedback.
6. SQL learning recommendations.
7. FastAPI documentation and the local server startup terminal.

Actual screenshots should be captured from the running application; none are fabricated or embedded in this document.

11. Challenges and Solutions

Challenge	Implemented response
Static and template paths depended on the working directory	Resolve directories relative to `main.py` using `BASE_DIR`
Template rendering API changed	Use the current request-first `TemplateResponse` call
Configured model became unavailable	Make the primary model configurable and add fallback identifiers
Quota exhaustion and temporary overload	Recognize selected provider errors and attempt another configured model
Model wrapped quiz JSON in Markdown fences	Clean matching fences before parsing JSON
Heavy local explanation model complicated setup	Keep local dependencies optional and provide Gemini fallback

These solutions improve usability but do not eliminate external service limits or malformed AI output.

12. Limitations

1. **External dependency:** Gemini requires internet access, a valid key, available model access, and quota. Model fallback remains within the same provider.
2. **Accuracy:** Responses, quiz answers, resource suggestions, and timelines may be incomplete or incorrect. No independent fact-checking or retrieval system is implemented.
3. **Latency and concurrency:** Synchronous generation is called inside asynchronous routes, which can block other work in that server process. There is no application-level end-to-end deadline or streaming output.
4. **Basic validation:** The backend lacks complete typed request schemas, input-length bounds, and strict quiz item validation.
5. **Error responses:** Provider errors are commonly embedded in HTTP 200 responses, making automated monitoring less reliable.
6. **Access control:** No authentication or application-level rate limiting is implemented. Anyone able to access the running application can consume the configured API quota.
7. **Frontend robustness:** Duplicate submissions are not disabled while waiting. Quiz options are built using HTML strings; the escaping helper does not escape quotes used in HTML attributes and needs hardening.
8. **No persistence:** Study history, scores, and learning progress are not stored. Learning paths are topic-based suggestions rather than profile-based adaptation.
9. **Optional local model:** LaMini inference is implemented but not verified in the current environment; large model downloads are unsuitable for the lightweight cloud workflow.
10. **Operational coverage:** There is no committed automated regression suite or comprehensive availability monitoring.

13. Future Enhancements

Reliability and usability

- Add typed input schemas, clear HTTP error codes, and strict quiz validation.
- Use structured model output where supported.
- Add request timeouts, controlled retries, and non-blocking AI calls.
- Disable duplicate form submissions and display clearer progress information.
- Render model content through safe DOM construction and use accessible form labels.
- Use multiline passage inputs and optional safe Markdown formatting.

Learning features

- Save study sessions and track quiz performance with a database.
- Personalize recommendations using learner-selected level and goals.
- Add multilingual and voice-based interaction.
- Support PDF and image input with appropriate extraction workflows.
- Introduce progress dashboards, spaced revision, and teacher tools.

Operations and evaluation

- Add authentication, per-user limits, and caching where appropriate.
- Create automated tests for endpoints, provider failures, and quiz rendering.
- Evaluate answer quality on a reviewed educational question set.
- Add monitoring and improve management of backend credentials.
- Evaluate local open models if hardware resources and privacy requirements justify their use.

These items are proposals and are not implemented features of the current version.

14. Conclusion

EduGenie demonstrates how a lightweight web application can combine several educational AI tasks in one interface. FastAPI provides the backend, JavaScript connects the forms to the APIs, and Gemini produces the requested content. Separating feature logic into modules makes the system understandable and easier to extend.

The project achieves its prototype goal of supporting questions, explanations, summaries, quizzes, and learning recommendations. Its usefulness depends on model quality and provider availability. Stronger validation, testing, concurrency handling, and learner-focused personalization are the next steps toward a more dependable educational service.

15. References

- FastAPI documentation: https://fastapi.tiangolo.com/
- Uvicorn documentation: https://www.uvicorn.org/
- Jinja documentation: https://jinja.palletsprojects.com/

- Google Gemini API documentation: https://ai.google.dev/gemini-api/docs
- Google Gemini rate limits: https://ai.google.dev/gemini-api/docs/rate-limits
- Google AI Studio: https://aistudio.google.com/
- Google Gen AI Python SDK: https://github.com/googleapis/python-genai
- Hugging Face Transformers: https://huggingface.co/docs/transformers/
- LaMini model page: https://huggingface.co/MBZUAI/LaMini-Flan-T5-783M
- MDN Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- Project repository: https://github.com/Maxiii-17/EduGenic

16. Demonstration and Viva Guide

Suggested five-minute demonstration

1. Explain the problem and show the homepage.
2. Ask the largest-ocean question.
3. Explain a familiar concept, such as Photosynthesis.
4. Summarize a short educational passage.
5. Generate a quiz and demonstrate both feedback outcomes.
6. Show a learning path and the API documentation.
7. Conclude with the architecture and key limitations.

Prepare the application and verify provider availability before the presentation. Actual generation time may make the demonstration longer; label any previously generated examples as such.

Common questions and answers

What is EduGenie?
A web-based assistant combining five educational tasks using generative AI.

Why use FastAPI?
It provides a concise Python API framework, ASGI support, and generated API documentation.

What is the role of Gemini?
It generates the text responses and quiz content from the supplied prompts. The application integrates a pretrained model rather than training a new one.

How is the quiz checked?
The browser compares the selected option text with the returned answer text and displays feedback.

Where is the API key stored?
In the local `.env` file loaded by the backend, rather than in the HTML page.

Does the application learn from each student?
No. Each feature request is independent; the current version does not store a learner profile or retrain the model.

Can it work offline?
The current main workflow requires Gemini connectivity. Only the optional LaMini explanation path could run locally after its dependencies and model files are installed.

Does fallback mean unlimited AI access?
No. It can improve availability when another configured model is accessible, but quota and provider restrictions still apply.

What is the main improvement you would make next?
Improve reliability with validated inputs and quiz outputs, appropriate error statuses, request deadlines, and automated tests.